Aston University
BIRMINGHAM UK

College of Engineering & Physical Sciences
Assignment Brief

AM41DP Data Science Programming	Main Coursework
Philip Trevelyan	p.trevelyan@aston.ac.uk

Assignment Brief / Coursework Content:

• This coursework consists of **two** tasks, with the second task involving a real world full data exploration activity, including exploratory data analysis, data pre-processing and clean-up. Freely available data sets from the real world will be provided, together with a set of specific objectives.

• Most of the coursework activities will feel familiar to students who have engaged with the lectures and tutorials. The activities will require some independent exploration and critical thinking, but the core concepts are covered in the lectures.

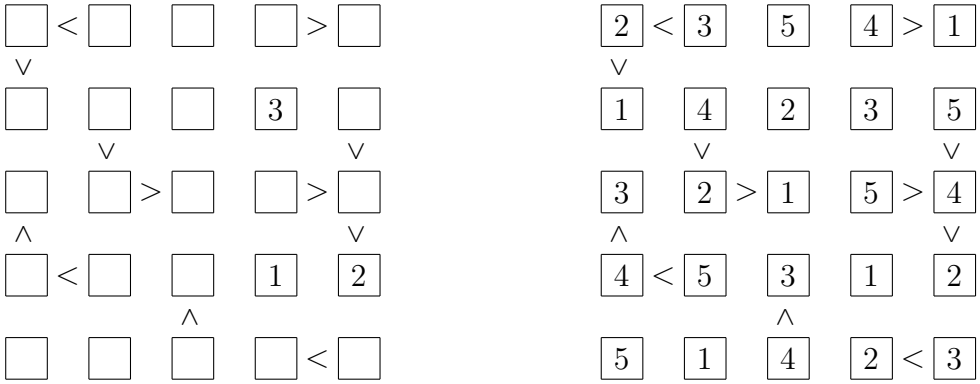
• The coursework must be completed individually. This does not mean that you cannot discuss with your peers or exchange ideas and insights, but only that your solution and report need to reflect your own thinking and understanding

• The coursework will assess all 4 Learning Outcomes of this module, using both **Python 3** and **SQL** for a variety of data science-related activities.

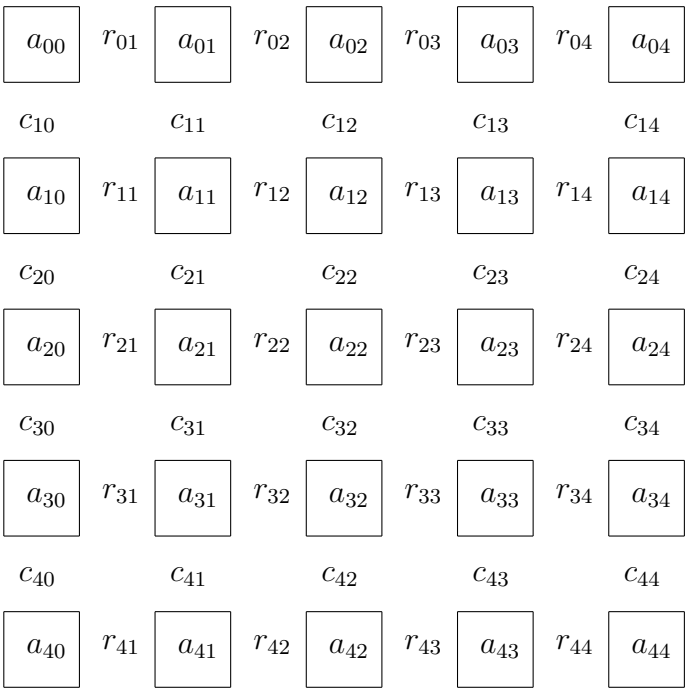
Task 1 - Python Coding (24%)

Consider a 5 by 5 grid containing some inequality signs and possibly some numbers. The objective is to fill in the rest of the grid using the following rules:
(i) Every row and column contains the number 1 to 5
(ii) Each row contains each number once
(iii) Each column contains each number once
(iv) Ensure that all of the inequality conditions are met

For example the initial problem and final solution could be:



This problem will be represented using the arrays a, r and c, arranged as follows:



The numbers in the grid are stored in a.
Throughout our algorithm each element a_{ij} will be represented by the set of possible elements it could be, initialised to $\{1, 2, 3, 4, 5\}$.
For clarity we use the word 'element' to denote the entire set in a_{ij}, and the word 'item' to denote a single number in a set.

The type of inequality is encoded using -1, 0 or 1.
Inequalities within the rows are defined using

$$r = \begin{cases} -1, & \text{represents } <, \text{ meaning the left is less than the right,} \\ 0, & \text{means there is no condition,} \\ 1, & \text{represents } >, \text{ meaning the left is greater than the right.} \end{cases}$$

Inequalities within the columns are defined using

$$c = \begin{cases} -1, & \text{represents } <, \text{ meaning the top is less than the bottom,} \\ 0, & \text{means there is no condition,} \\ 1, & \text{represents } >, \text{ meaning the top is greater than the bottom.} \end{cases}$$

Write a Python 3 code to do the following:

(A) **Initial:** Create a function with no inputs that outputs a, r and c such that:
(i) Set every element of an array $a[0..4, 0..4]$ to be the set $\{1, 2, 3, 4, 5\}$.
(ii) Set every value in an array $r[0..4, 1..4]$ to zero.
(iii) Set every value in an array $c[1..4, 0..4]$ to zero.

(B) **Total:** Create a function to output the total number of items in a.

(C) **Uniqueness:** Create a function with input a:
If an element in a only contains one item, then remove this item from the other elements in the same row and column.

(D) **Row Inequalities:** Create a function with inputs a and r:
(i) If $r[i, j] = 1$ then: [This means $a[i, j-1] > a[i, j]$]
Remove all items in element $a[i, j-1] \leq$ minimum item in element $a[i, j]$.
Remove all items in element $a[i, j] \geq$ maximum item in element $a[i, j-1]$.
(ii) If $r[i, j] = -1$ then: [This means $a[i, j-1] < a[i, j]$]
Remove all items in element $a[i, j-1] \geq$ maximum item in element $a[i, j]$.
Remove all items in element $a[i, j] \leq$ minimum item in element $a[i, j-1]$.

(E) **Column Inequalities:** Create a function inputs a and c:
(i) If $c[i, j] = 1$ then: [This means $a[i-1, j] > a[i, j]$]
Remove all items in element $a[i-1, j] \leq$ minimum item in element $a[i, j]$.
Remove all items in element $a[i, j] \geq$ maximum item in element $a[i-1, j]$.
(ii) If $c[i, j] = -1$ then: [This means $a[i-1, j] < a[i, j]$]
Remove all items in element $a[i-1, j] \geq$ maximum item in element $a[i, j]$.
Remove all items in element $a[i, j] \leq$ minimum item in element $a[i-1, j]$.

(F) **Pairs:** Create a function with input a:
(i) If a row contains 2 elements which are equal and only contain 2 items then:
Remove these 2 items from the three remaining elements in that row.

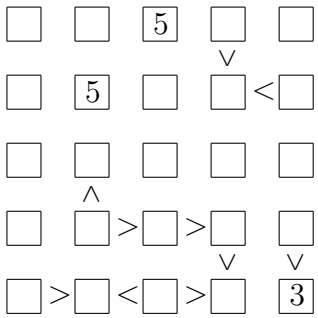
(ii) If a column contains 2 elements which are equal and only contain 2 items:
Remove these 2 items from the three remaining elements in that column.

(G) **Triples:** Create a function with input a:
(i) If a row contains 3 elements which are equal and only contain 3 items then:
Remove these 3 items from the two remaining elements in that row.
(ii) If a column contains 3 elements which are equal and only contain 3 items:
Remove these 3 items from the two remaining elements in that column.

(H) **Existence:** Create a function with input a:
(i) If an item only occurs once in a row then:
Make the element with this item be a set only with this item.
(ii) If an item only occurs once in a column then:
Make the element with this item be a set only with this item.

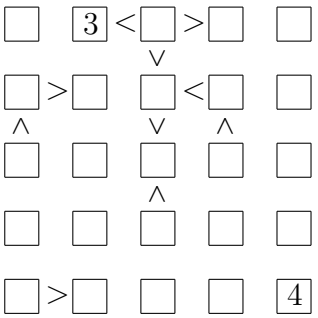
(I) **Main:** Create a function with inputs a, r and c and does the following:
Let T1 equal the value given by your function in (B).
Let T2 equal 126.
Write to the screen the number zero and T1.
Create a loop to iterate over the following whilst T1>25:
Call the functions in parts (C) to (H).
Let T1 equal the value given by your function in (B).
Write to the screen the iteration number and T1.
Write suitable error statements for T1>125, or T1<25 or T1=T2.
Now let T2 equal T1.
If T1=25 the program should terminate and display the solution for a.

(J) **Configuration:** Write 4 functions to encode each of these initial configurations:



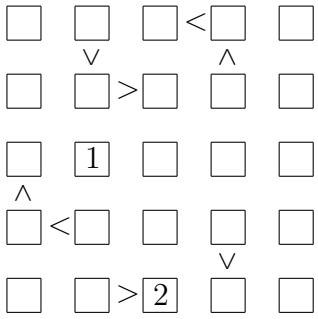
Problem 1

$a_{02} = \{5\}$, $a_{11} = \{5\}$, $a_{44} = \{3\}$,
$r_{14} = -1$, $r_{32} = 1$, $r_{33} = 1$,
$r_{41} = 1$, $r_{42} = -1$, $r_{43} = 1$,
$c_{13} = 1$, $c_{31} = -1$, $c_{43} = 1$, $c_{44} = 1$.



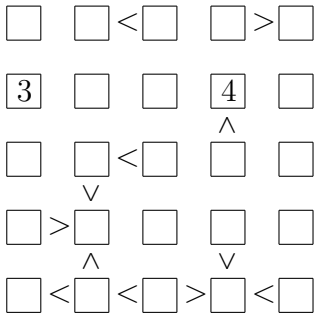
Problem 2

$a_{01} = \{3\}$, $a_{44} = \{4\}$,
$r_{02} = -1$, $r_{03} = 1$, $r_{11} = 1$, $r_{13} = -1$,
$r_{41} = 1$, $c_{12} = 1$, $c_{20} = -1$, $c_{22} = 1$,
$c_{23} = -1$, $c_{32} = -1$.



Problem 3

$a_{21} = \{1\}$, $a_{42} = \{2\}$,
$r_{03} = -1$, $r_{12} = 1$, $r_{31} = -1$, $r_{42} = 1$,
$c_{11} = 1$, $c_{13} = -1$, $c_{30} = -1$, $c_{43} = 1$.



Problem 4

$a_{10} = \{3\}$, $a_{13} = \{4\}$,
$r_{02} = -1$, $r_{04} = 1$, $r_{22} = -1$, $r_{31} = 1$,
$r_{41} = -1$, $r_{42} = -1$, $r_{43} = 1$, $r_{44} = -1$,
$c_{23} = -1$, $c_{31} = 1$, $c_{41} = -1$, $c_{43} = 1$.

(K) Now you can test you code:
First run the function in (A) to define a, r and c.
Second run one of the initial configuration functions in (J).
Finally run the function in (I).

(L) Can you think of any insufficiencies with this algorithm?
How would you go about modifying this algorithm to a 7 by 7 grid?

[Don't do this, just say how you would do it].

Your code will probably need to include some of the following Python commands:

and	as	break	def	dtype	elif	empty	for
if	in	int	import	len	log10	max	min
np	numpy	object	print	range	remove	return	sum
while	zeros	+	-	=	**	<=	>=
==	!=	[	]	{	}	(	)
:	;	”					

Please try to avoid using any built-in functions. If you use any other built in functions you may get a lower mark.

Task 2 – Predicting an Election (76%)

You will need to download the csv file 'Election_results.csv' from Blackboard, which came from https://commonslibrary.parliament.uk/research-briefings/cbp-8647/
It contains 18348 rows each representing the results of a single seat in a UK election from 1918 to 2024. There are 18 columns named:

'constituency_id',	'constituency_name',	'country/region',	'electorate',
'con_votes',	'con_share',	'lib_votes',	'lib_share',
'lab_votes',	'lab_share',	'natSW_votes',	'natSW_share',
'oth_votes',	'oth_share',	'total_votes',	'turnout',
'election',	'boundary_set'		

Write Python code in Jupyter to do parts (a)-(h) and SQL code to do part (i).

(a) Read in the csv file 'Election_results.csv' to create a dataframe. Remove the column named 'boundary_set'. Fill in any missing values with zeros. Remove any rows where 'total_votes' = '-1'. Remove any rows where the columns 'electorate', 'lib_votes' or 'lab_votes' equal a space, i.e. ' '. Set the 6 columns 'electorate', 'lib_votes', 'lab_votes', 'con_votes', 'natSW_votes' and 'oth_votes' to be type integer. Create a column called 'Result' which equals one of these 5: 'con', 'lab', 'lib', 'nat' or 'oth' depending on which of these 5 parties got the greatest share of the vote, using the 5 columns 'con_share', 'lab_share', 'lib_share', 'natSW_share', and 'oth_share'. Remove any rows where 'Result' is not equal to one of 'con', 'lab', 'lib', 'nat' or 'oth'. How many rows and columns do you have? (10 marks)

(b) Using **Matplotlib**, create a bar chart showing the total number of seats for the 5 parties 'con', 'lab', 'lib', 'nat' or 'oth'. Comment on any dominant features in the bar chart. (4 marks)

(c) Using **Matplotlib**:

(i) Create a Histogram using 'con_votes' for the number of Conservative votes against frequency using 50 bins.

(ii) Create a Histogram using 'lab_votes' for the number of Labour votes against frequency using 50 bins.

(iii) Create a Histogram using 'electorate' for the number of eligible voters against frequency using 50 bins.

Comment on the distributions of the histograms. (6 marks)

(d) Using Matplotlib, create a scatterplot of 'con_share' against 'lab_share' using 5 different colours to illustrate the 'Result'. Use blue for 'con', red for 'lab', yellow for 'lib', green for 'nat' and black for 'oth'. Comment on any special dividing lines that appear in the scatterplot. (6 marks)

(e) Remove the 10 columns 'country/region', 'constituency_id', 'turnout', 'con_votes', 'lib_votes', 'lab_votes', 'oth_votes', 'natSW_votes', 'total_votes' and 'electorate'. Create a list called 'election_list' of the 29 different elements in 'election'. Create 5 columns 'con_av', 'lab_av', 'lib_av', 'nat_av' and 'oth_av' which equal the corresponding mean values of 'con_share', 'lab_share', 'lib_share', 'natSW_share', and 'oth_share' when 'election' equals each value in the list 'election_list'.

Create a list called 'constituency_list' of the 2300 different elements in 'constituency_name'. Create 5 columns 'con_bias', 'lab_bias', 'lib_bias', 'nat_bias' and 'oth_bias' which equal the corresponding mean values of 'con_share', 'lab_share', 'lib_share', 'natSW_share', and 'oth_share' when 'constituency_name' equals each value in the list 'constituency_list'. Create 5 columns defined by
'con_diff' = 'con_bias' − 'con_av'
'lab_diff' = 'lab_bias' − 'lab_av'
'lib_diff' = 'lib_bias' − 'lib_av'
'nat_diff' = 'nat_bias' − 'nat_av'
'oth_diff' = 'oth_bias' − 'oth_av'
Obtain the pairwise correlation matrix of your data frame, determine any correlations above 0.89 or below −0.52, and discuss any significant correlations. (10 marks)

(f) You are now going to use the 15 columns 'con_av', 'lab_av', 'lib_av', 'nat_av', 'oth_av', 'con_bias', 'lab_bias', 'lib_bias', 'nat_bias', 'oth_bias', 'con_diff', 'lab_diff', 'lib_diff', 'nat_diff', 'oth_diff' to predict the 'Result' column using logistic regression in sklearn. Split the data so that all the elections before 2024 are your training data and the 2024 election is the test data. Using sklearn, predict the 'Result' column. Test the accuarcy of your model using the test data set. Make a classification report and confusion matrix from your models predictions on the test data. Determine how many seats each of 'con', 'lab', 'lib', 'nat' and 'oth' won in the actual and models prediction for the test data. Discuss the accuracy of your results. (8 marks)

(g) You are now going to use the same 15 columns 'con_av', 'lab_av', 'lib_av', 'nat_av', 'oth_av', 'con_bias', 'lab_bias', 'lib_bias', 'nat_bias', 'oth_bias', 'con_diff', 'lab_diff', 'lib_diff', 'nat_diff', 'oth_diff' to predict the 'lab_share' column using linear regression in sklearn. Split the data so that all the elections before 2024 are your training data and the 2024 election is the test data. Using sklearn, predict the 'lab_share' column. Test your model on the test data and using sklearn obtain the 'Mean Absolute Error', 'Mean Squared Error' and 'R-squared'. Next, create a scatter plot of predicted lab share against the actual lab share. Discuss the errors and limitations of your model. (8 marks)

(h) Remove any rows where the 'Result' is not one of these 3 parties 'con', 'lab' or 'lib'. Now remove the 12 columns 'con_share', 'lib_share', 'lab_share', 'natSW_share', 'oth_share', 'constituency_name' 'nat_av', 'oth_av', 'nat_bias', 'oth_bias', 'nat_diff', and 'oth_diff'. Next make a copy of the data frame. Create a list of the 3 unique values in the Result column. Now create a column called 'Result_number' which takes on the values 0, 1 or 2 when the 'Result' is 'con', 'lab' or 'lib', respectively. Now remove the 'Result' column. Next normalise the 9 columns 'con_av', 'lab_av', 'lib_av', 'con_bias', 'lab_bias', 'lib_bias', 'con_diff', 'lab_diff', and 'lib_diff', so that each column has a minimum of 0 and a maximum of 1. Now split the data so that all the elections before 2024 are your training data and the 2024 election is the test data. You are now going to use the 9 columns 'con_av', 'lab_av', 'lib_av', 'con_bias', 'lab_bias', 'lib_bias', 'con_diff', 'lab_diff', and 'lib_diff', to predict the 'Result_number' column using PyTorch. Create any classification model you like using PyTorch; select an appropriate criterion, optimisation algorithm, and learning rate. Test the accuracy of your model using the test data set. Make a classification report and confusion matrix from your models predictions on the test data. Determine how many seats each con, lab and lib won in the actual and models prediction for the test data. Discuss the accuracy of your model. (16 marks)

(i) Using SQLite Studio. Read in the csv file 'Election_results.csv' to create a table. Remove the column named 'boundary_set'. Remove any rows where 'total_votes' = '-1'. Remove any rows where the columns 'electorate', 'lib_votes' or 'lab_votes', equal a space, i.e. ' '. Create a column called 'Result' which equals one of these 5: 'con', 'lab', 'lib', 'nat' or 'oth' depending on which of these 5 parties got the greatest share of the vote, using the 5 columns 'con_share', 'lab_share', 'lib_share', 'natSW_share', and 'oth_share'. How many rows and columns do you have? (8 marks)

Other solution Guidelines:

This coursework involves both **Python** and **SQL**. Please submit Task 1 and Task 2(a)-(h) in Python as 2 separate **Jupyter Notebook** files (i.e. files ending in ipynb) and submit Task 2(i) in **SQL** as a text file. Carry out your CW solution process in a systematic way, and include comments in your code. Don't forget to label the axes, include colours and legends when necessary, and give a title to each figure.

Marking:
The weighting for this coursework are as follows:

Description	Weight
Task 1 – Python Coding	24%
Task 2 – Predicting an Election	76%

Key Dates:

Coursework set	Friday 18 October 2024
Submission date	2pm UK time Friday 20 December 2024
Expected feedback return date (individual coursework feedback summary, available on Blackboard.)	TBC

Mark allocation in each part includes accuracy, efficiency and presentation.

Submission Details:
Coursework files must be submitted electronically on Blackboard, using the link available in the 'Assessment' area. Late submissions will be given a 10% penalty per day.

Report submission guide:

• Please clearly indicate your name in your codes.

• The report will be submitted through a specific link on Blackboard, which will be located under Assessment - Coursework.

• You need to submit 3 programming files: two Jupyter Notebook (ipynb) files and a text file containing your commands in SQL.

• The files must be able to be parsed assuming that the original CSV data files are available under the same folder as the report file.

• Reports submitted as any document format other than the ones above will be subject to a flat penalty of 5 marks. Do not compress your files.

• The total length of the report (excluding template section headers, cover sheet and references) must not exceed 50 pages in total. (You can do a 'print preview' to check). A 10 mark penalty will be given if the page count exceeds this.